

Learn Low-Level Design in 25 Days: Day 5—How to Think in Objects (Turning Requirements into...

1. The Developer’s Dilemma

TechPreneur

Published October 17, 2025

Free: No

Read on Freedium · original

Contents

1. The Developer's Dilemma 0

2. Thinking in Objects — The Core Idea 0

3. From Requirements to Classes — Step-by-Step 0

Step 1: Read Requirements Like a Detective 0

Step 2: Find Relationships 0

Step 3: Define Responsibilities 0

Step 4: Model It in Code 0

4. Visual Explanation 0

5. Case Study: WhatsApp Chats 0

6. Try It Yourself 0

7. Key Takeaways 0

8. What's Next 0

1. The Developer's Dilemma

Picture this: your PM drops a new feature request on your desk —

> "We need a ride-tracking system like Uber. It should show drivers, rides, and real-time updates."

You nod. Sounds simple enough… until you open your IDE. Where do you even start? A `RideManager` class? A `User` class? Should `Driver` extend `User`? What about `Location` or `TripHistory`?

This is the moment where most developers freeze — not because they don't know code, but because they don't yet know how to think in objects.

Welcome to **Day 5 of "Learn Low-Level Design in 25 Days"** — where we turn vague requirements into meaningful, well-modeled classes. By the end of this post, you'll be able to look at any requirement and break it down into interacting objects that mirror real-world behavior — the foundation of great object-oriented design.



(You can follow the full 25-day series here: Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8))

2. Thinking in Objects — The Core Idea

When we design systems, our job isn't to "write code." It's to **model the real world inside a computer**.

Every software system represents something tangible — a food delivery app models restaurants, orders, riders; a banking app models accounts, transactions, ledgers.

So the trick is:

> "Identify the nouns and verbs from requirements, and then organize them into objects that behave like real things."

Let's see how.

3. From Requirements to Classes — Step-by-Step

Step 1: Read Requirements Like a Detective

Requirement:

> A user can book a ride. The system assigns a nearby driver. The ride has start and end locations, and once completed, the fare is calculated.

Your brain should start identifying nouns and verbs:

NounsVerbsUser, Driver, Ride, Location, Farebook, assign, start, end, calculate

That's your first clue. Each noun is a potential **class**. Each verb is a **method** or **interaction**.

Step 2: Find Relationships

Now, think about how these nouns connect.

- A **User** can **book** a **Ride**.
- A **Driver** is **assigned** to a **Ride**.

- A **Ride** has **Locations** (start, end).

Let's visualize that:

```
User ----> Ride <---- Driver
             |
             +--> Location (start, end)
```

Already, you're forming a mental picture of your system.

Step 3: Define Responsibilities

Every class should own one clear responsibility.

ClassResponsibilityUserRequest ridesDriverAccept ridesRideManage ride state (requested, ongoing, completed)LocationHold coordinatesFareCalculatorCompute fare based on distance/time

See how we introduced a **FareCalculator**? That's a behavioral helper object — it doesn't represent a person or thing, but it encapsulates logic cleanly.

Step 4: Model It in Code

Here's how it might look in Python-like pseudocode:

```python
class Location:
    def __init__(self, lat, lng):
        self.lat = lat
        self.lng = lng

class User:
    def __init__(self, name):
        self.name = name

    def request_ride(self, start, end):
        return Ride(self, start, end)

class Driver:
    def __init__(self, name, location):
        self.name = name
        self.location = location

class Ride:
    def __init__(self, user, start, end):
        self.user = user
        self.start = start
        self.end = end
        self.driver = None
        self.status = "REQUESTED"

    def assign_driver(self, driver):
        self.driver = driver
        self.status = "ONGOING"

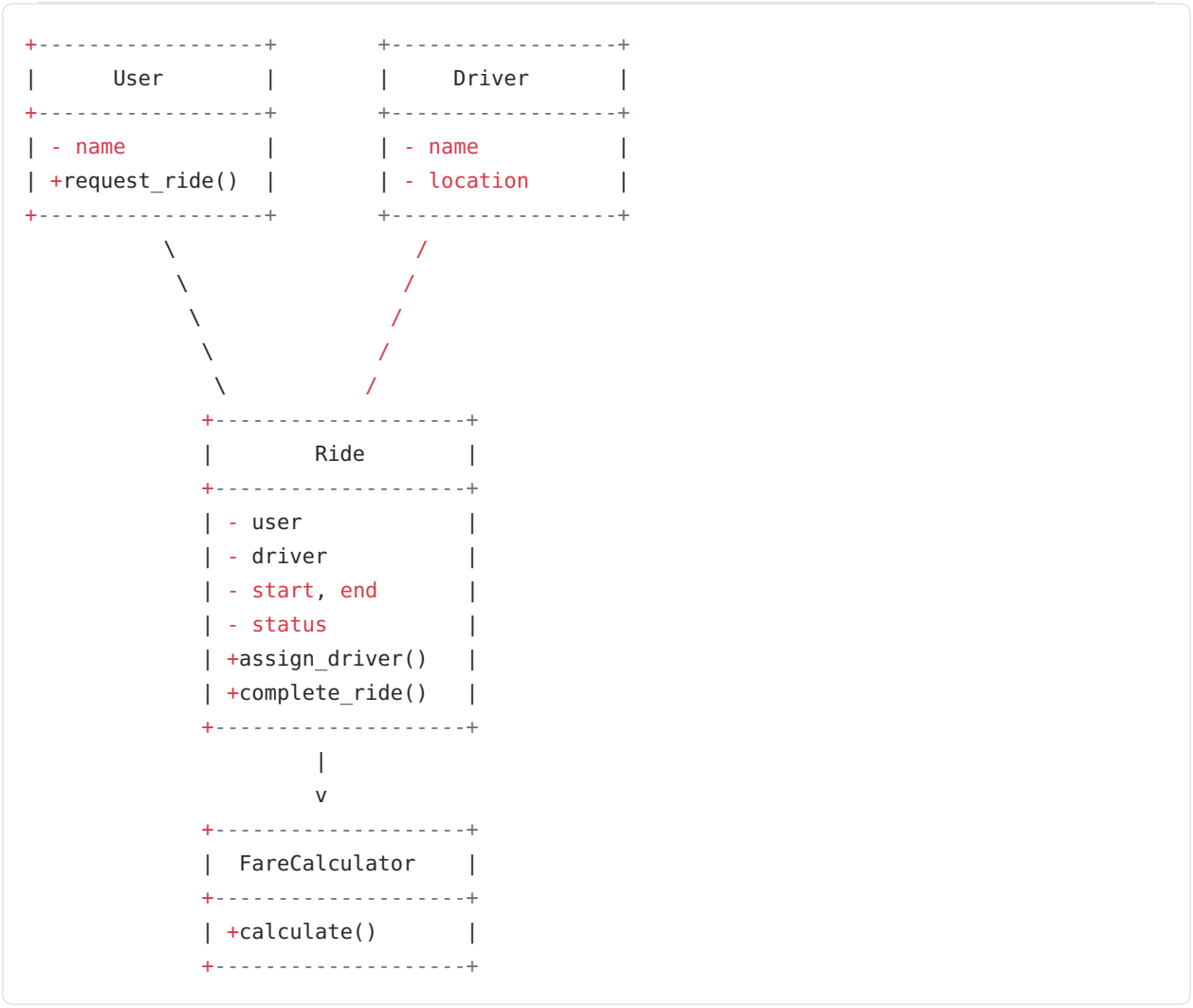
    def complete_ride(self):
        self.status = "COMPLETED"
        return FareCalculator.calculate(self.start, self.end)

class FareCalculator:
    @staticmethod
    def calculate(start, end):
        # pretend distance-based fare
        return 100
```

Now you can almost see the system running in your head.

4. Visual Explanation

Here's a simple class relationship diagram:



This is what it means to "think in objects."

5. Case Study: WhatsApp Chats

Even something as simple as WhatsApp chat follows this logic:

- **User** sends a **Message** in a **Chat**.
- Each **Chat** can have multiple **Messages** and **Participants**.
- A **Message** has **sender**, **timestamp**, and **content**.

Turn that into classes, and you've already modeled 70% of WhatsApp's backend conversation logic.

6. Try It Yourself

Challenge 1: Pick a simple system — like a Library Management System or Online Food Ordering App. Write down the nouns and verbs. Turn them into classes and methods.

Challenge 2: Identify one "helper" class (like `FareCalculator`) you'd extract to reduce duplication.

7. Key Takeaways

- Think in **nouns (classes)** and **verbs (methods)**.
- Keep one **responsibility per class** — it's the heart of clean LLD.
- Relationships matter more than individual classes.
- Use **helper classes** to encapsulate logic.
- Visualize your design before you code.

When you master this mindset, code becomes a natural expression of design — not the other way around.

8. What's Next

Tomorrow, we'll get a little more visual. **Day 6 — UML Basics for Developers: Turning Designs into Diagrams That Communicate.**

We'll take what you modeled today and turn it into clean, professional UML diagrams that even non-engineers can understand.

(Save and follow the full series here: Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8))